Serialization and Relationships : Code-Along (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p4-v2-serialization-relationships)  (https://github.com/learn-co-curriculum/python-p4-v2-serialization-relationships/issues/new)

Learning Goals

- Use SQLAlchemy-Serializer to convert SQLAlchemy objects with relationships into dictionaries.

Key Vocab

- **Serialization**: a process to convert programmatic data such as a Python object to a sequence of bytes that can be shared with other programs, computers, or networks.
- **Deserialization**: the reverse process, converting a sequence of bytes back to programmatic data.
- **SQLAlchemy-Serializer**: A powerful tool for serializing data in Python using the SQLAlchemy ORM.

Introduction

We saw in a prior lesson how to use SQLAlchemy-Serializer to convert a SQLAlchemy model into a dictionary. In this lesson, we'll serialize models that have relationships. We'll use serialization rules to include or exclude attributes to avoid issues with nested models and infinite recursion.

Setup

This lesson is a code-along, so fork and clone the repo.

Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

Create and seed the database:

```
flask db init
flask db migrate -m 'initial migration'
flask db upgrade head
python seed.py
```

Updating the code to serialize the data will not impact the database schema; we won't need to touch Flask-Migrate again in this lesson.

SerializerMixin

Navigate to `models.py` and you'll notice the import at the top:

```
# models.py
from sqlalchemy_serializer import SerializerMixin
```

The SerializerMixin class in SQLAlchemy-Serializer is a helpful feature that allows developers to quickly add serialization capabilities to their SQLAlchemy models. When a model class inherits from the SerializerMixin, it gains a range of methods for serializing and deserializing data. These methods include `to_dict()` , which converts the model object into a dictionary, and `to_json()` , which converts it into a JSON string.

In short, the SerializerMixin simplifies the process of data serialization by adding a set of predefined methods to SQLAlchemy models. Developers can customize these methods as needed to achieve their desired serialization format, and can use them to quickly transform complex database models into simpler, more usable data structures. Most languages can't work with Python objects, after all.

Configuring our Models for Serialization

In `models.py`, we'll need to reconfigure each of our models to inherit from `SerializerMixin`. Don't worry though- this only requires a small amount of new code, and we won't have to run new migrations afterward.

```
# models.py
# imports, config

class Zookeeper(db.Model, SerializerMixin):
    ...

class Enclosure(db.Model, SerializerMixin):
    ...

class Animal(db.Model, SerializerMixin):
    ...
```

By now you should have created your database and run `seed.py`; if you haven't yet, do that now!

Once you have a populated database, navigate to the `server/` directory and run `flask shell` to start manipulating our models. Import all of your models and retrieve a `Zookeeper` record. Let's run its brand new method, `to_dict()`:

```
$ from models import *
$ z1 = Zookeeper.query.first()
$ z1.to_dict()
# => RecursionError: maximum recursion depth exceeded in comparison
```

While this isn't *quite* what we were looking for, it introduces us to an important concept in serialization: *recursion depth*.

Recursion Depth

Sometimes, the process of serialization can get very complex, especially if the data we're working with has many layers of nested structures or relationships.

Recursion depth refers to how deeply we traverse the nested relationships within data structures when we serialize them. If the structures are very deeply nested, a complete serialization process can require a lot of memory and computational resources, which can slow down the program or even cause it to crash.

For example, imagine you have a data structure representing a family tree, with each person having parents, grandparents, and so on. If we try to serialize this structure and we don't set a limit on the recursion depth, the program might keep going deeper and deeper into the family tree, creating more and more data to process, until it runs out of memory or crashes.

To avoid this problem, we can set a limit on the recursion depth, so that the program only goes a certain number of layers deep before stopping. This helps us manage the memory and computational resources needed for the serialization process and ensures that the program runs smoothly.

`serialize_rules`

To avoid any errors involving recursion depth, we can set the `serialize_rules` class attribute in each of our models. This is a tuple (so remember to include trailing commas!) where we can specify which fields to exclude. To avoid diving too many layers into each record's relationships, we will tell SQLAlchemy-Serializer to not look back at the original record from within its related records. Here's what that will look like:

```
# models.py
# imports, config

class Zookeeper(db.Model, SerializerMixin):
    __tablename__ = 'zookeepers'

    # don't forget that every tuple needs at least one comma!
    serialize_rules = ('-animals.zookeeper',)
```

```
class Enclosure(db.Model, SerializerMixin):
    __tablename__ = 'enclosures'

    serialize_rules = ('-animals.enclosure',)

class Animal(db.Model, SerializerMixin):
    __tablename__ = 'animals'

    serialize_rules = ('-zookeeper.animals', '-enclosure.animals',)
```

Save your changes and navigate back to the Flask shell. Let's try converting our record to a dictionary again (your result will differ):

```
$ z1 = Zookeeper.query.first()
$ z1.to_dict()
# => {'birthday': '1961-08-19', 'id': 1, 'name': 'Christina Hill', 'animals': [{'name': 'Heather', 'enclosure_id': 1
```

Just like that, we have a dictionary representation of a Python SQLAlchemy object. This will be much easier for other applications to use!

serialize_only

We can also use the `serialize_only` class attribute on a model to specify fields to include in serialization. For example, we can serialize the zookeeper's id and name, along with the name and species of each associated animal (don't modify your code, this is just an example of `serialize_only`).

```
class Zookeeper(db.Model, SerializerMixin):
    __tablename__ = 'zookeepers'
    serialize_only = ('id', 'name', 'animals.name', 'animals.species',)
    serialize_rules = ()

    id = db.Column(db.Integer, primary_key=True)
```

```
    name = db.Column(db.String, unique=True)
    birthday = db.Column(db.Date)

    animals = db.relationship('Animal', back_populates='zookeeper')
```

Calling `to_dict()` on the zookeeper results in the following:

```
>>> z1 = Zookeeper.query.first()
>>> z1.to_dict()
{'id': 1, 'animals': [{'species': 'Elephant', 'name': 'Paul'}, {'species': 'Hippo', 'name': 'Jennifer'}, {'species':
>>>
```

to_dict()

`to_dict()` is a simple method: it takes a SQLAlchemy object, turns its columns into dictionary keys, and turns its column values into dictionary values. That being said, it can do a bit more if we ever need to modify its output.

`to_dict()` has two arguments that can be passed in:

- `rules` works the same as `serialize_rules` within the model. You can specify additional columns to exclude here.
- `only` allows you to specify an exhaustive list of columns to display. This can be helpful if you're working with a table with many columns or you only want to display one or two of a table's columns.

Let's head back to the Flask shell and give these a shot:

```
$ z1 = Zookeeper.query.first()
$ z1.to_dict(rules=('-animals',))
# => {'name': 'Christina Hill', 'id': 1, 'birthday': '1961-08-19'}
$ z1.to_dict(only=('name',))
# => {'name': 'Christina Hill'}
```

Instructions

After configuring serialization on each of your models, run `pytest` to ensure that each is serializable to a dictionary without any errors. When all tests are passing, submit your work through CodeGrade.

Conclusion

SQLAlchemy-Serializer is a helpful tool that helps programmers turn complex database information into simpler, portable formats. It makes it easier to share this data with other programs or systems. For instance, if you have a list of friends on Facebook, SQLAlchemy-Serializer can help you turn that data into a format that another website or app can understand.

However, when we serialize data, it can sometimes become too complex and cause problems. To prevent this, programmers need to set limits on how deep the data can go. For example, imagine a list of animals with each animal having offspring, and each of those offspring having their own offspring. The list could go on forever! SQLAlchemy-Serializer helps programmers manage this issue by providing tools such as `serialize_rules` and the `rules` and `only` arguments to the `to_dict()` method to handle these kinds of situations.

By using SQLAlchemy-Serializer, programmers can create faster and more efficient programs that can easily share data with others.

Solution Code

```
# server/models.py

from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import MetaData
from sqlalchemy_serializer import SerializerMixin

convention = {
    "ix": "ix_%(column_0_label)s",
```

```
        "uq": "uq_%(table_name)s_%(column_0_name)s",
        "ck": "ck_%(table_name)s_%(constraint_name)s",
        "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",
        "pk": "pk_%(table_name)s"
    }

    metadata = MetaData(naming_convention=convention)

    db = SQLAlchemy(metadata=metadata)


    class Zookeeper(db.Model, SerializerMixin):
        __tablename__ = 'zookeepers'

        # don't forget that every tuple needs at least one comma!
        serialize_rules = ('-animals.zookeeper',)

        id = db.Column(db.Integer, primary_key=True)
        name = db.Column(db.String, unique=True)
        birthday = db.Column(db.Date)

        animals = db.relationship('Animal', back_populates='zookeeper')


    class Enclosure(db.Model, SerializerMixin):
        __tablename__ = 'enclosures'

        serialize_rules = ('-animals.enclosure',)

        id = db.Column(db.Integer, primary_key=True)
        environment = db.Column(db.String)
        open_to_visitors = db.Column(db.Boolean)
```

```
    animals = db.relationship('Animal', back_populates='enclosure')


class Animal(db.Model, SerializerMixin):
    __tablename__ = 'animals'

    serialize_rules = ('-zookeeper.animals', '-enclosure.animals',)

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String, unique=True)
    species = db.Column(db.String)

    zookeeper_id = db.Column(db.Integer, db.ForeignKey('zookeepers.id'))
    enclosure_id = db.Column(db.Integer, db.ForeignKey('enclosures.id'))

    enclosure = db.relationship('Enclosure', back_populates='animals')
    zookeeper = db.relationship('Zookeeper', back_populates='animals')

    def __repr__(self):
        return f'<Animal {self.name}, a {self.species}>'
```

Resources

- **Quickstart - Flask-SQLAlchemy (https://flask-sqlalchemy.palletsprojects.com/en/2.x/quickstart/#)**
- **Flask-Migrate (https://flask-migrate.readthedocs.io/en/latest/)**
- **SQLAlchemy-Serializer (https://pypi.org/project/SQLAlchemy-serializer/)**

This tool needs to be loaded in a new browser window

Load Serialization and Relationships : Code-Along (CodeGrade) in a new window